UNIVERSIDAD
NACIONAL
DE COLOMBIA

Laboratory 6
Performance

Jeisson Andrés Vergara Vargas

Software Architecture

2026-I

---

i. Objective

The objective of this laboratory is to carry out a set of **performance tests**.

ii. Performance Tests

a. Requirements

- **Option 1:** JMeter tool.
- **Option 2:** k6 tool.

b. Component Deployment

Select a feature of the software system developed as the course project and **deploy** the following components:

- A database.
- A microservice.
- The API Gateway.
- The web front-end.

Note: the following activities must be carried out on a node completely independent from the infrastructure where the system components are deployed.

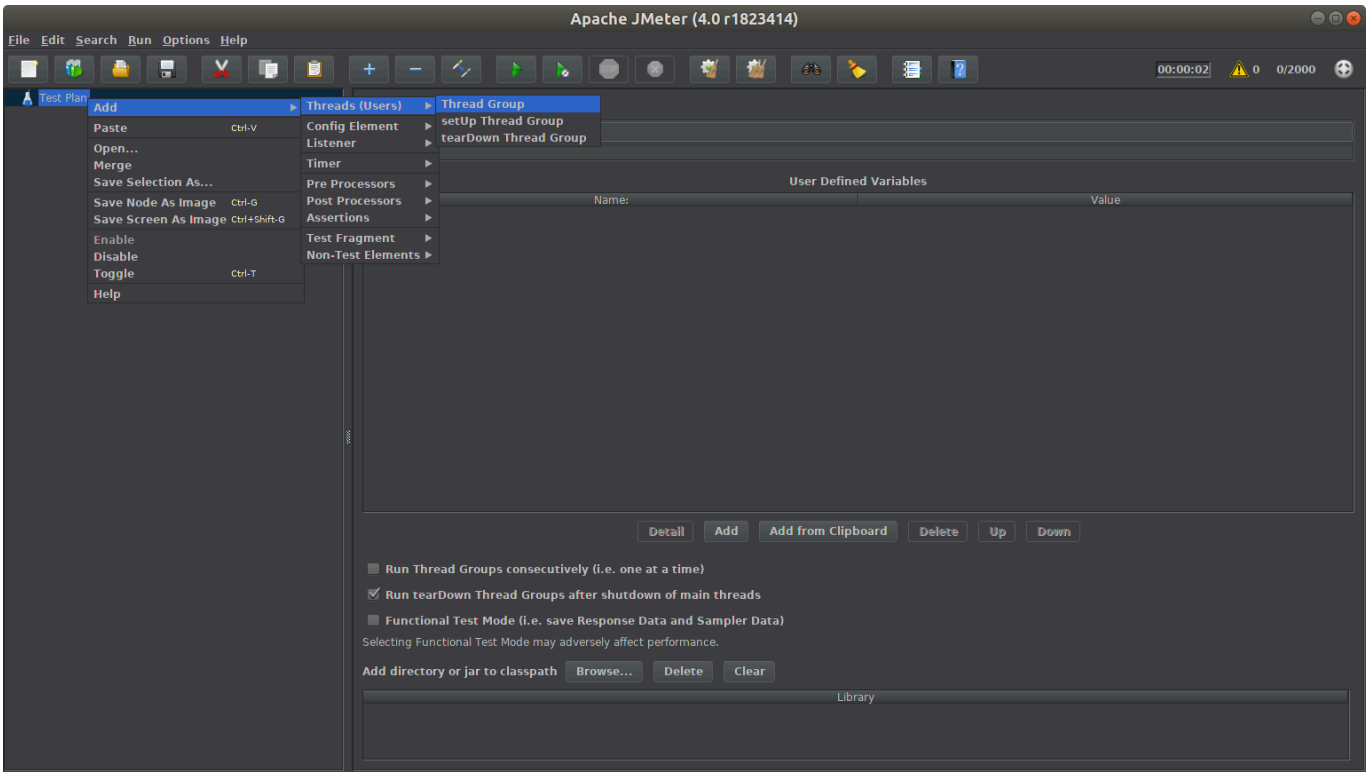
---

iii. Option 1: JMeter

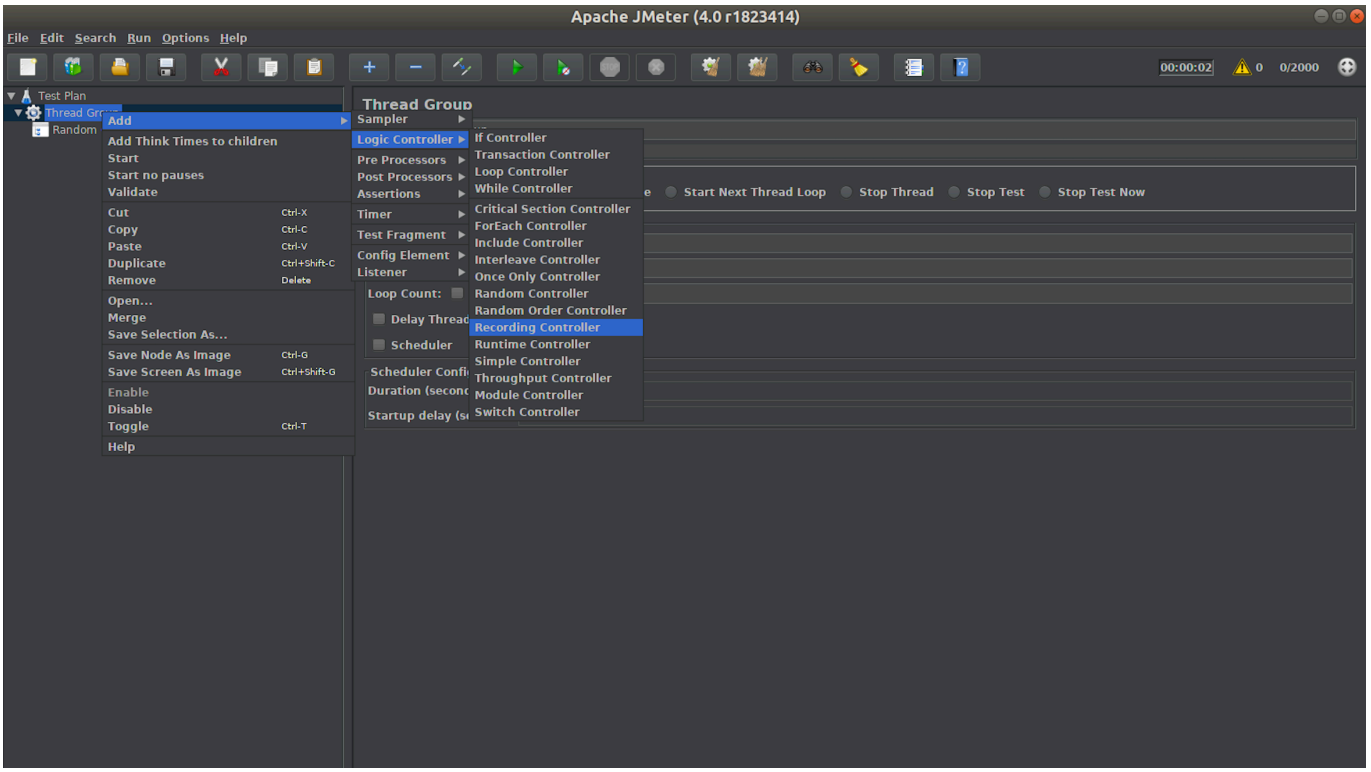
c. Test Environment Setup

1. Open the **JMeter** application.

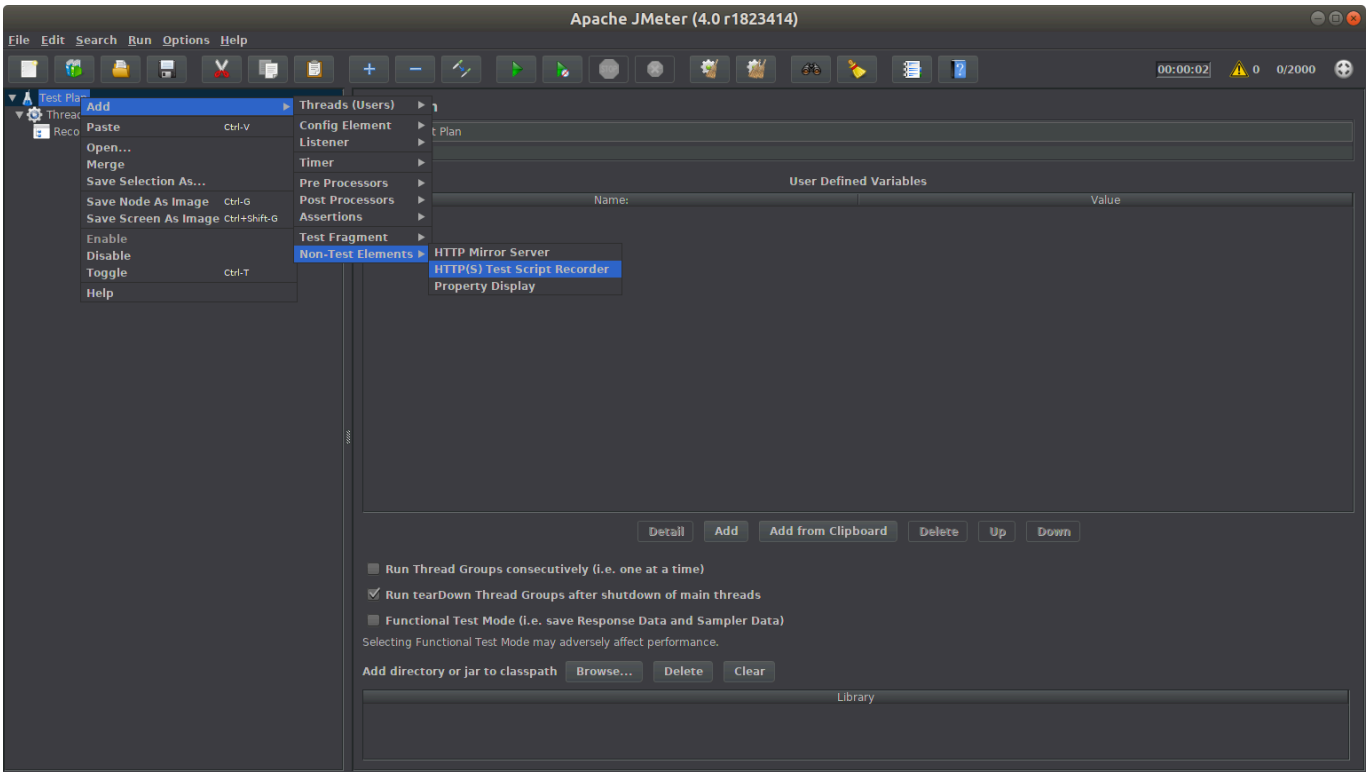
2. In the **Test Plan**, add a new **Thread Group**:



3. Inside the created **Thread Group**, add a new **Recording Controller**:



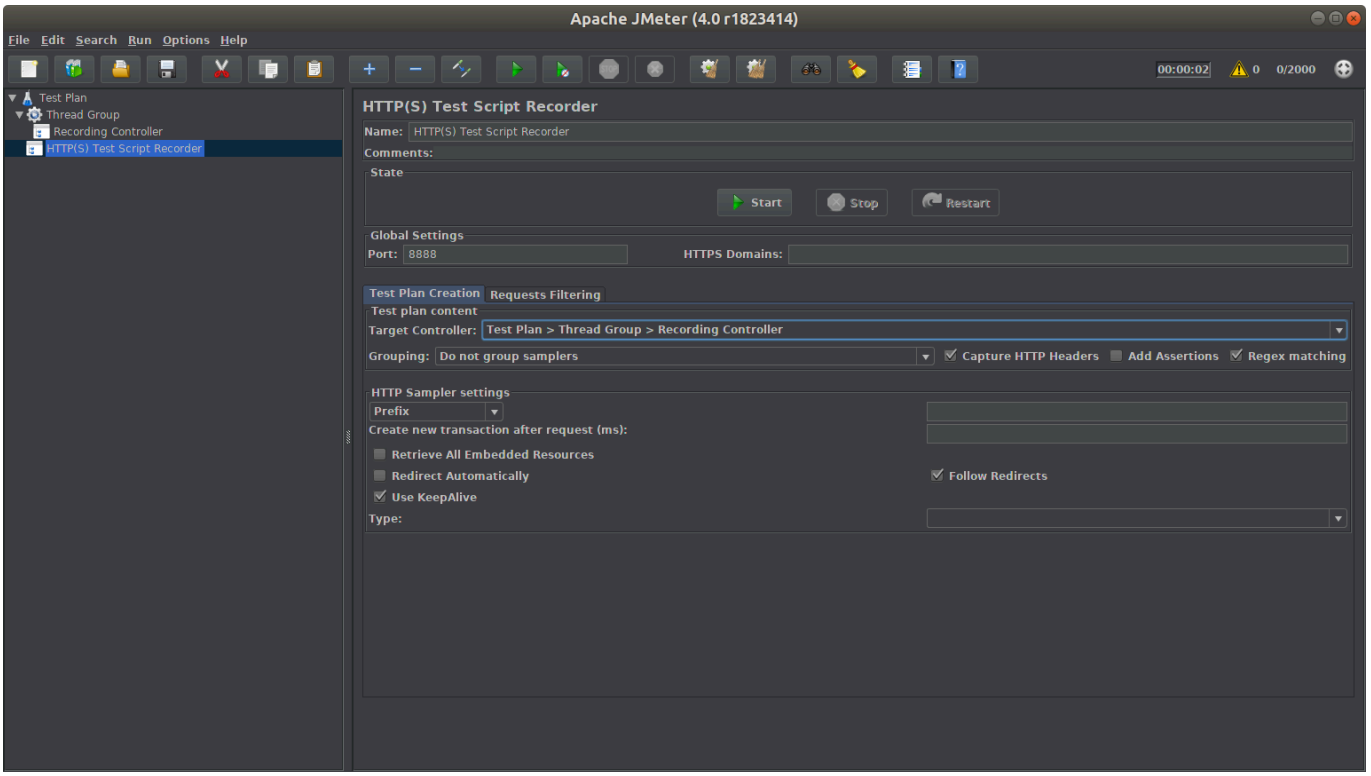
4. In the **Test Plan**, add a new **HTTP(s) Test Script Recorder**:

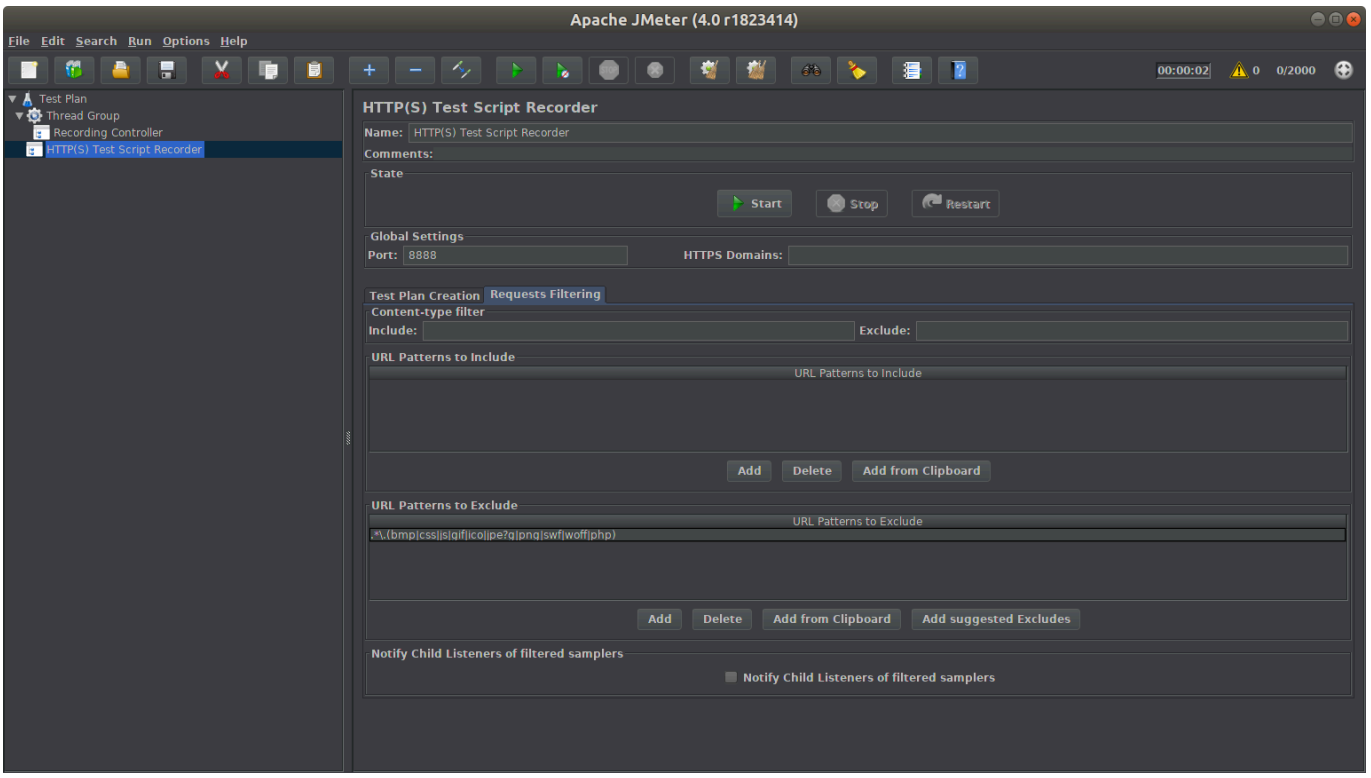


d. Feature Recording

1. Configure the added **HTTP(s) Test Script Recorder**:

```
* Port: 8888
* Target Controller: Test Plan > Thread Group > Recording Controller
* Capture HTTP Headers: enabled
* Regex matching: enabled
* URL Patterns to Exclude: .*\.(bmp|css|js|gif|ico|jpe?g|png|swf|woff|php)
```

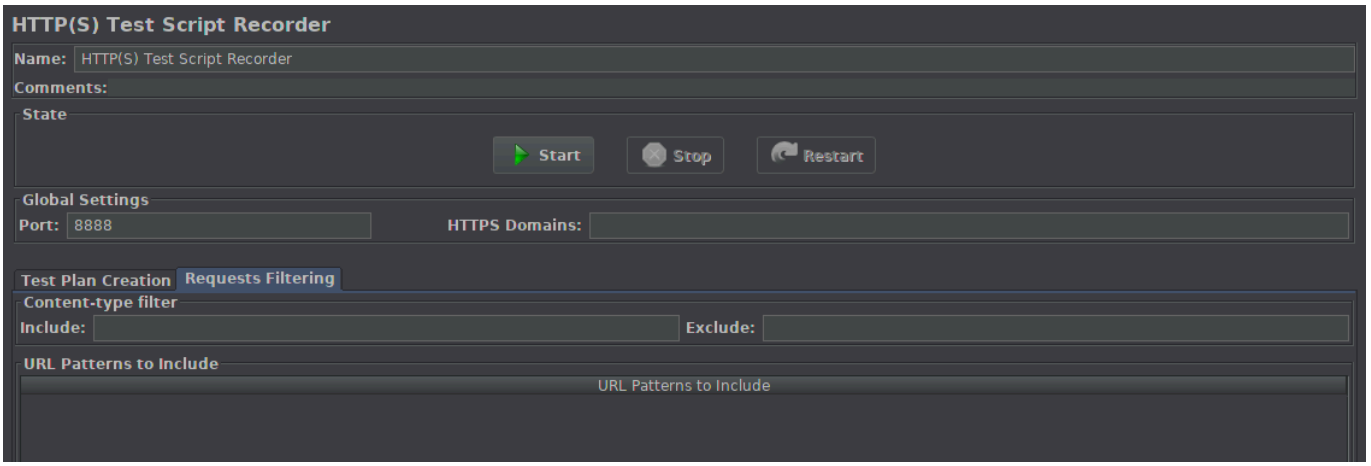


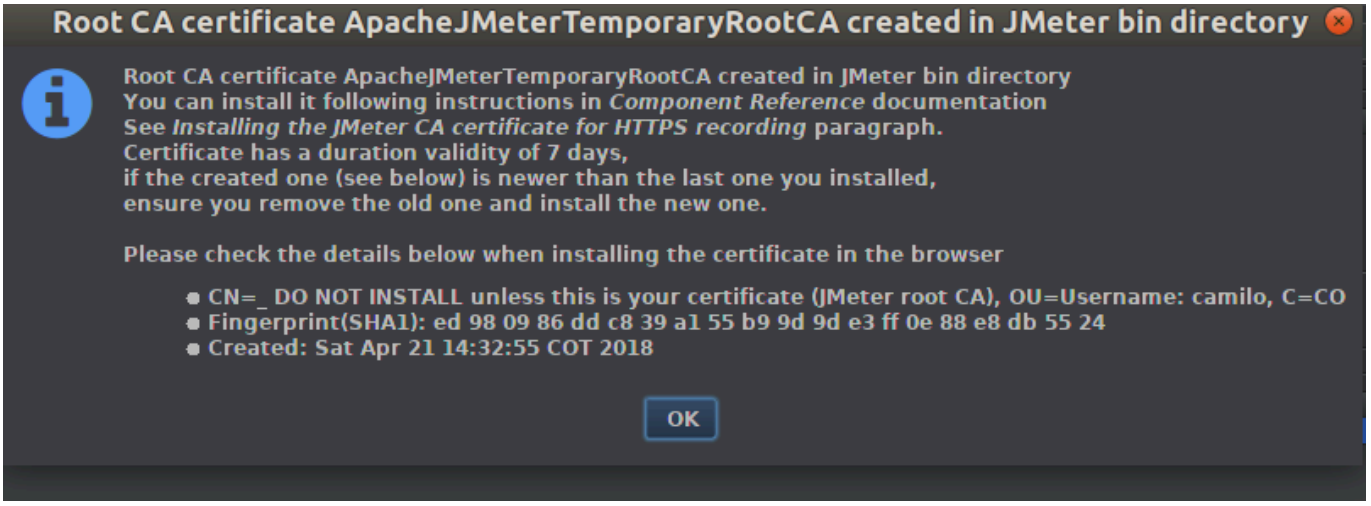


2. Open the **web front-end** in a browser. Verify the URI where this component is deployed.

3. Configure the **HTTP Proxy** in the browser where the application is open: *localhost:8888*.

4. Start the recording process in JMeter (**Start**):

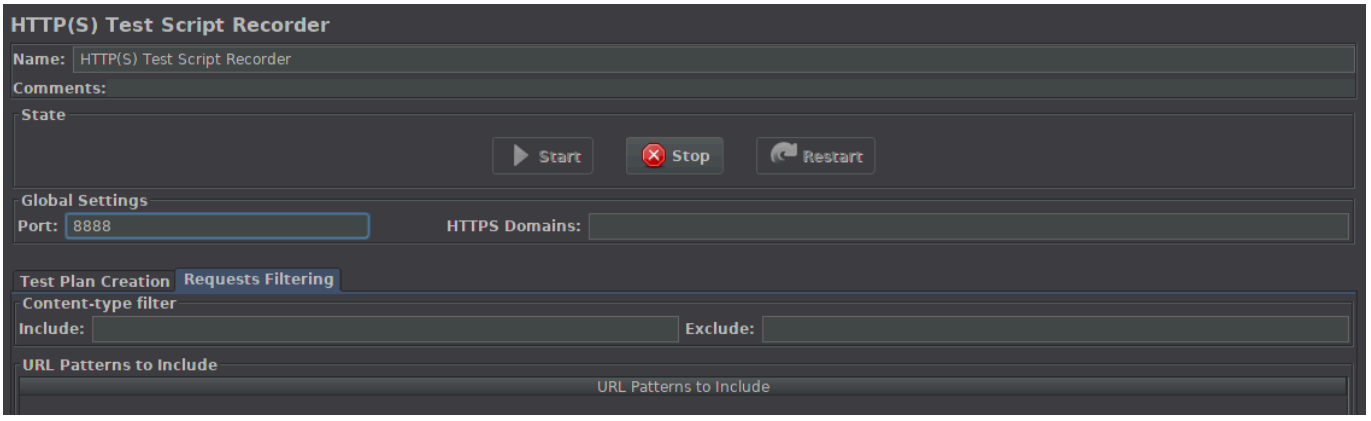




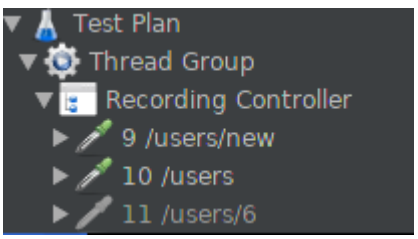
5. In the browser, refresh the page where the web front-end is open.

6. Use a feature that is supported by the deployed components.

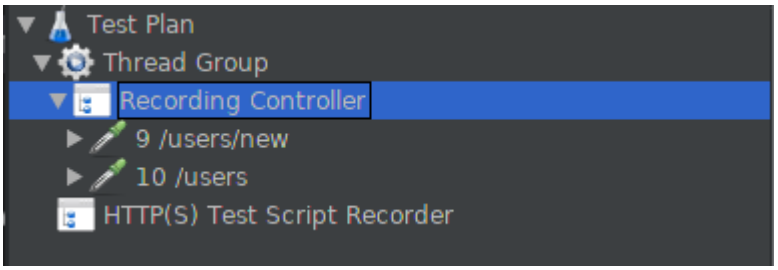
7. In JMeter, stop the recording process (**Stop**):



8. In the **Recording Controller**, the recorded test will be visible as a list of captured requests, similar to those shown in the image:

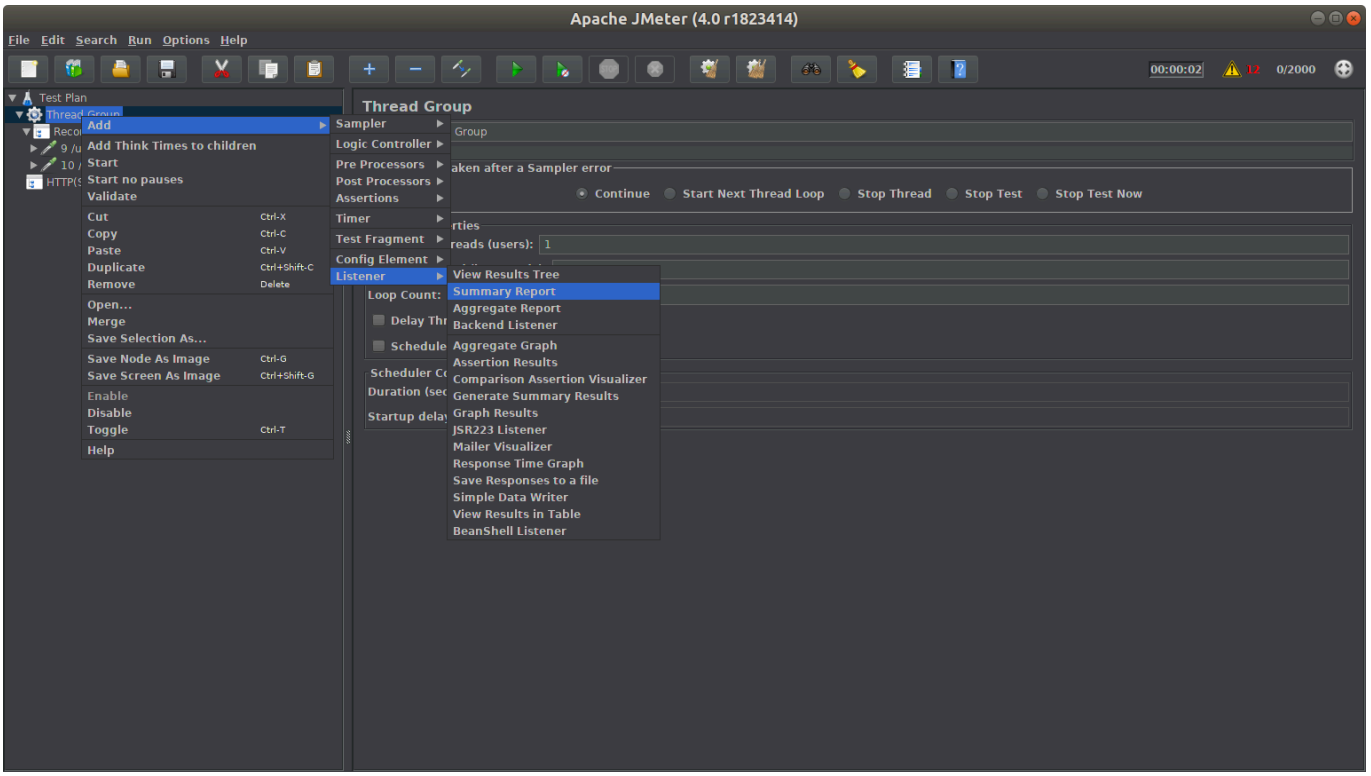


9. Remove any requests that are unrelated to the interaction with the executed feature:

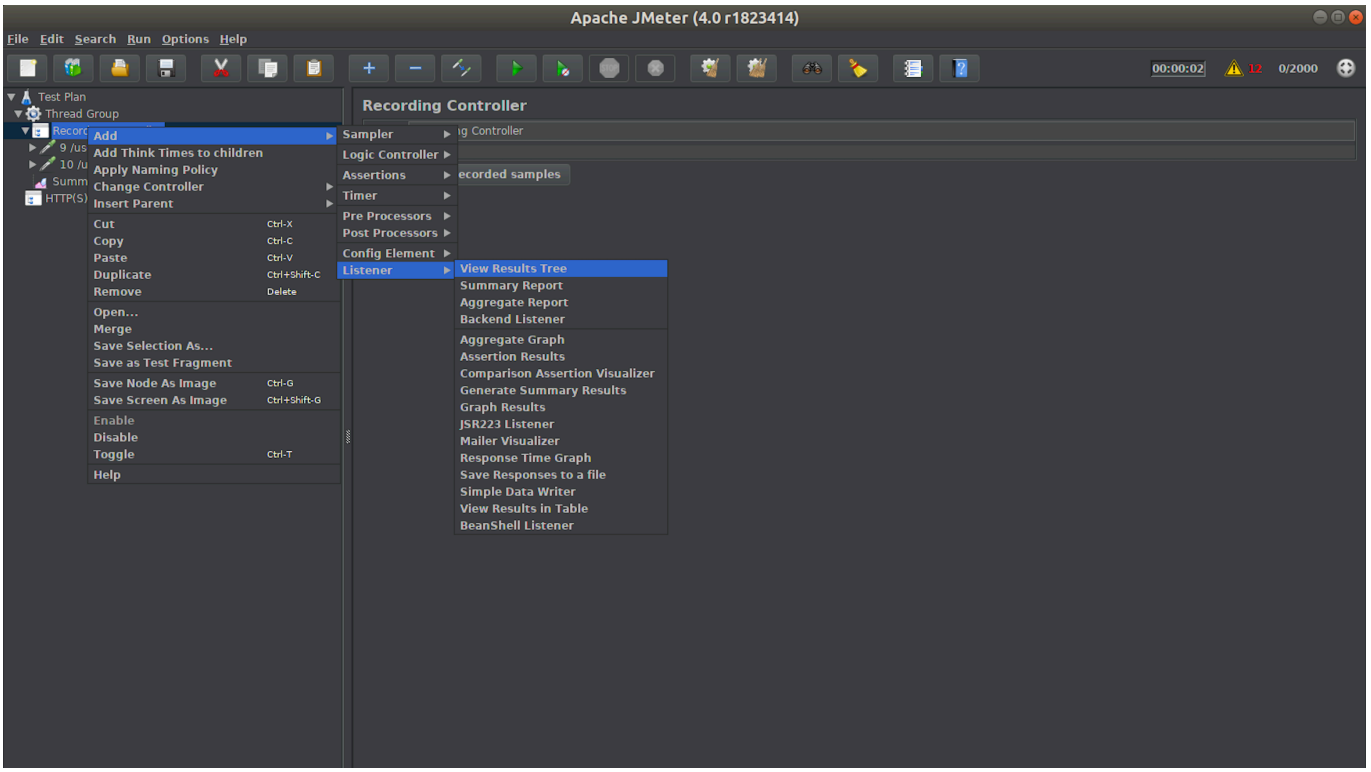


e. Recording Cleanup

1. In the **Thread Group**, add a new **Summary Report**:



2. In the **Recording Controller**, add a new **View Results Tree**:



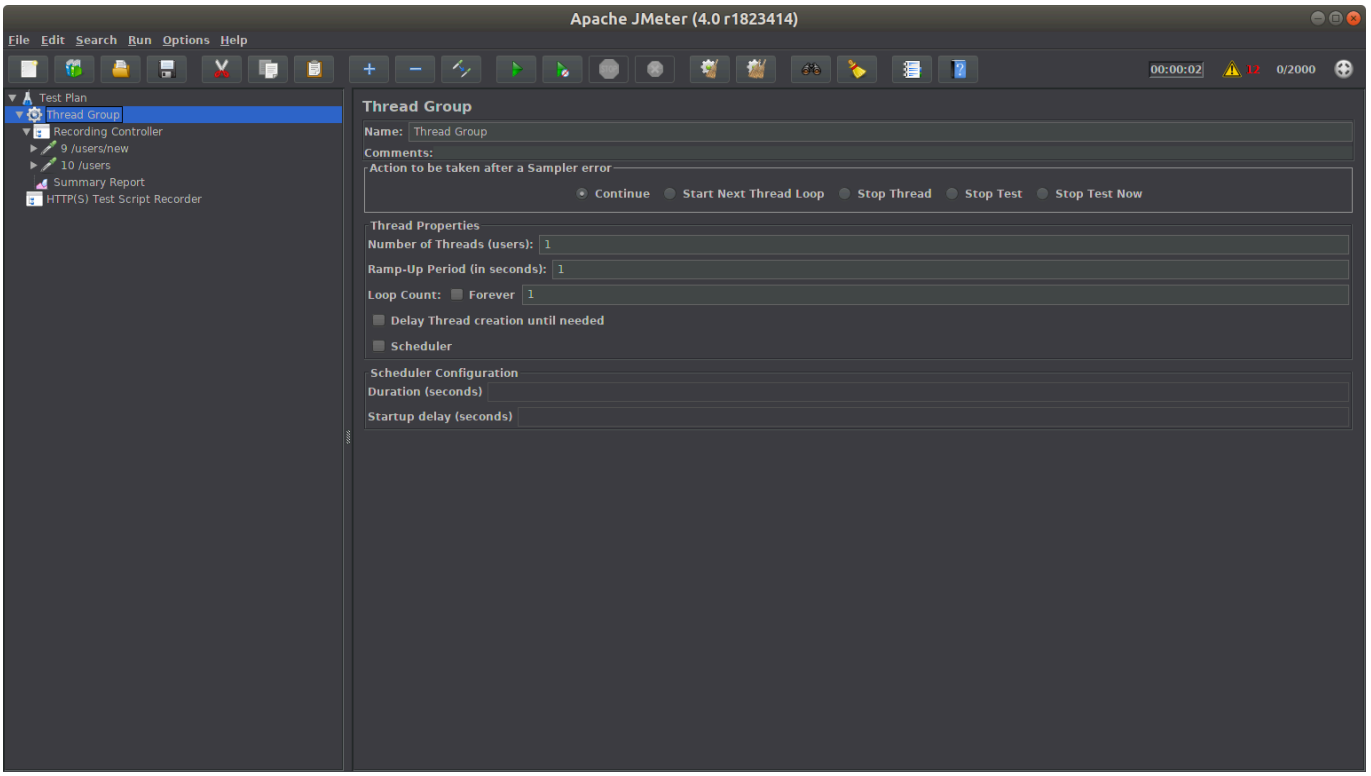
f. Load Test Execution

1. In the browser, remove the **HTTP Proxy** configuration set earlier.

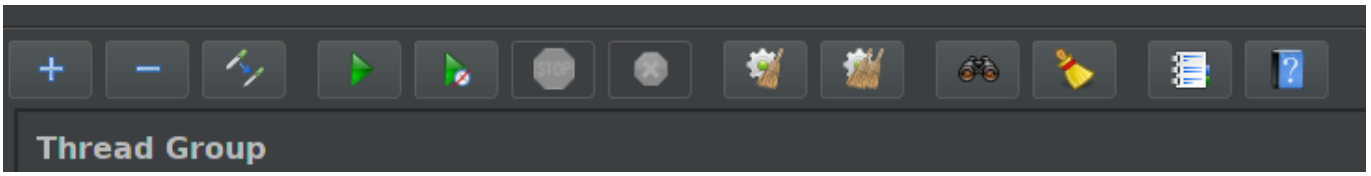
2. In the **Thread Group**, apply the following configuration:

- Number of Threads (users): **1**
- Ramp-Up Period (in seconds): **1**

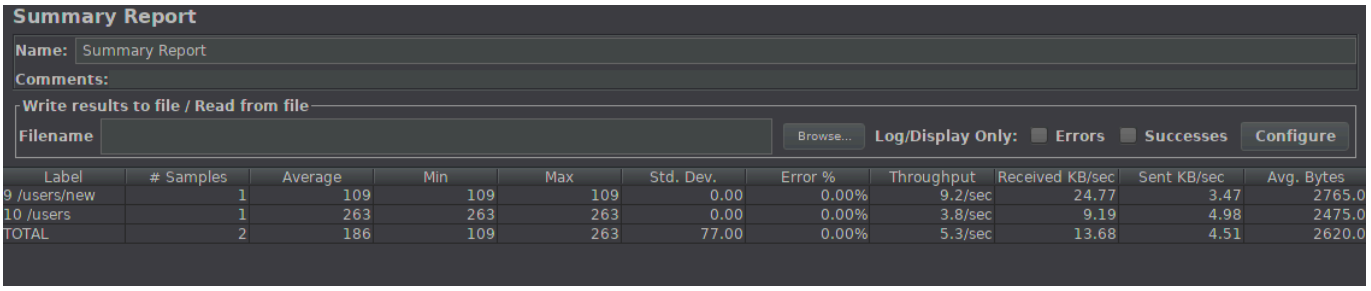
The **Number of Threads** represents the number of users that will access the system within the specified **Ramp-Up Period**. In this case, 1 user will access the system over a period of 1 second.



3. Run the test configured in the previous step (**Play**):



4. In the **Summary Report**, observe the test results. Note the response time (in milliseconds) of the system when the feature is executed. For example, in a sample run, a response time of 263 ms = 0.263 s was obtained for a single user making the request.



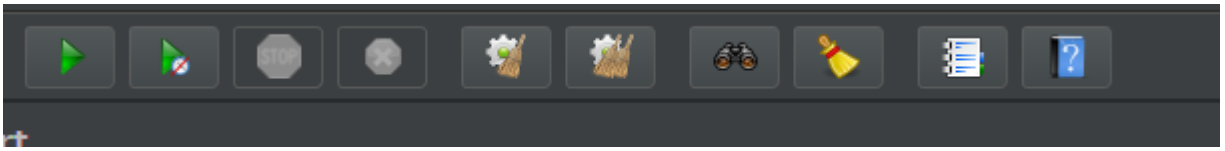
Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
9 /users/new	1	109	109	109	0.00	0.00%	9.2/sec	24.77	3.47	2765.0
10 /users	1	263	263	263	0.00	0.00%	3.8/sec	9.19	4.98	2475.0
TOTAL	2	186	109	263	77.00	0.00%	5.3/sec	13.68	4.51	2620.0

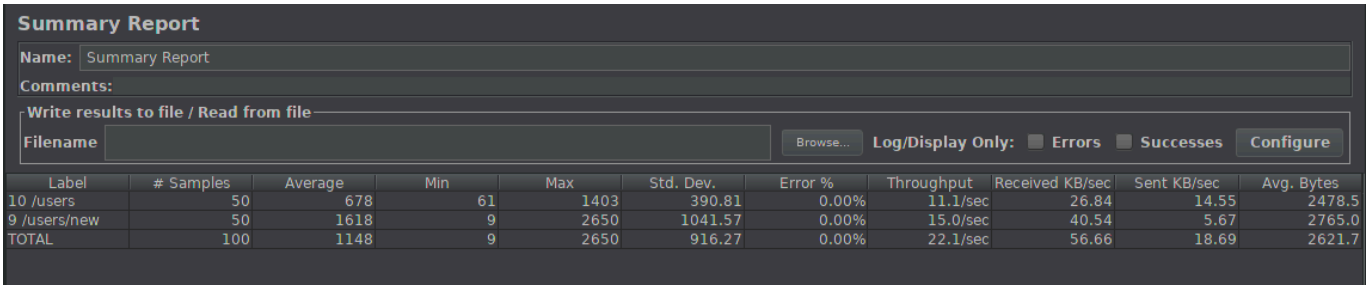
5. To run a test with **50 concurrent users**, apply the following configuration in the **Thread Group**:

- Number of Threads (users): **50**
- Ramp-Up Period (in seconds): **1**

6. In this case, 50 users will access the system over 1 second, at intervals of **Ramp-Up Period** / **Number of Threads** = 1s / 50 = 0.02s.

Note: before running the test, any trace of the previous test must be cleared using the clean action (**broom icon**):

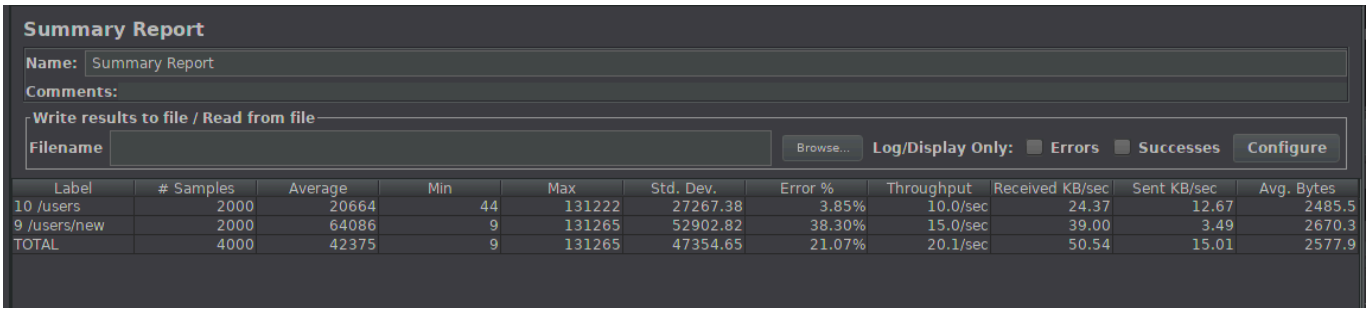




Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
10 /users	50	678	61	1403	390.81	0.00%	11.1/sec	26.84	14.55	2478.5
9 /users/new	50	1618	9	2650	1041.57	0.00%	15.0/sec	40.54	5.67	2765.0
TOTAL	100	1148	9	2650	916.27	0.00%	22.1/sec	56.66	18.69	2621.7

7. The **Summary Report** will display the results of the test. For example, in a sample run with 50 concurrent users, a response time of 1618 ms = 1.6 s was obtained.

8. With 2000 concurrent users, an error rate greater than 0 was observed, meaning the system was unable to respond to all requests at that load level.



Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
10 /users	2000	20664	44	131222	27267.38	3.85%	10.0/sec	24.37	12.67	2485.5
9 /users/new	2000	64086	9	131265	52902.82	38.30%	15.0/sec	39.00	3.49	2670.3
TOTAL	4000	42375	9	131265	47354.65	21.07%	20.1/sec	50.54	15.01	2577.9

Conclusion: the **knee** of the **performance curve** must be below 2000 concurrent users.

9. Build the **performance chart** using the data collected in the previous steps.

---

iv. Option 2: k6

c. k6 Installation

Install k6 according to the operating system:

- **Linux (Debian/Ubuntu):**

```
sudo gpg -k
sudo gpg --no-default-keyring --keyring /usr/share/keyrings/k6-
archive-keyring.gpg \
  --keyserver hkp://keyserver.ubuntu.com:80 --recv-keys
C5AD17C747E3415A3642D57D77C6C491D6AC1D69
echo "deb [signed-by=/usr/share/keyrings/k6-archive-keyring.gpg]
https://dl.k6.io/deb stable main" \
  | sudo tee /etc/apt/sources.list.d/k6.list
sudo apt-get update && sudo apt-get install k6
```

- **macOS:**

```
brew install k6
```

- **Windows:**

  See the documentation.

d. Test Script Creation

1. Verify the URI where the component to be tested is deployed (API Gateway or microservice).

2. Create a file named `performance_test.js` with the following base content, replacing `<API_GATEWAY_URI>` with the URI of the deployed component and `/endpoint` with the path corresponding to the selected feature:

```
import http from 'k6/http';
import { sleep, check } from 'k6';

export const options = {
  vus: 1,           // number of concurrent virtual users
  duration: '30s',  // test duration
};

export default function () {
  const res = http.get('http://<API_GATEWAY_URI>/endpoint');

  check(res, {
    'status is 200': (r) => r.status === 200,
  });

  sleep(1);
}
```

3. The `vus` (*Virtual Users*) parameter is equivalent to JMeter's **Number of Threads**: it represents the number of concurrent users that will send requests to the system simultaneously for the duration specified in `duration`.

e. Load Test Execution

1. Run the test with **1 concurrent user**:

```
k6 run performance_test.js
```

2. Once finished, k6 will display a metrics summary in the console. The main metric to observe is `http_req_duration`, which represents the response time in milliseconds — equivalent to the **Average** field in JMeter's Summary Report:

```
http_req_duration..............: avg=263ms  min=210ms  med=255ms  max=340ms
http_req_failed................: 0.00%
```

3. To run the test with **50 concurrent users**, update the `performance_test.js` file:

```
export const options = {
  vus: 50,
  duration: '30s',
};
```

Then run again:

```
k6 run performance_test.js
```

4. To sweep across different load levels and build the performance curve, it is recommended to use **stages**, which allow varying the number of users over time within a single test execution:

```
export const options = {
  stages: [
    { duration: '30s', target: 1    },  // 1 user
    { duration: '30s', target: 50   },  // 50 users
    { duration: '30s', target: 200  },  // 200 users
    { duration: '30s', target: 500  },  // 500 users
    { duration: '30s', target: 2000 },  // 2000 users
    { duration: '30s', target: 0    },  // ramp-down
  ],
};
```

5. With 2000 concurrent users, the `http_req_failed` field is expected to show an error rate greater than 0, indicating that the system is unable to handle all requests at that load level.

```
http_req_duration..............: avg=4521ms
http_req_failed................: 12.34%
```

Conclusion: the **knee** of the **performance curve** must be below the user threshold at which `http_req_failed` exceeds 0%.

6. Build the **performance chart** using the `http_req_duration` (avg) values collected for each load level tested.

Note: for both options, the data collected in the steps above must be used to identify the **knee of the performance curve** of the system under test.